

Topic 7: Loops — for, while, and do-while

6.1 Why Loops?

- Loops execute a block of code repeatedly while a condition holds
- Without loops: printing numbers 1–100 would require 100 lines
- With a loop: 3 lines handle it

6.2 The for Loop

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";  
}  
// Output: 1 2 3 4 5
```

- Three parts: initialization, condition, update — all in one line
- Use when you know exactly how many iterations are needed
- The loop variable (i) is local to the for loop

6.3 The while Loop

```
int i = 1;  
while (i <= 5) {  
    cout << i << " ";  
    i++;  
}
```

- Condition checked before each iteration — may not run at all if false initially
- Use when number of iterations depends on a condition, not a fixed count
- Always ensure the condition eventually becomes false — otherwise infinite loop

6.4 The do-while Loop

```
int choice;  
do {  
    cout << "Enter 1 to continue, 0 to exit: ";
```

```
    cin >> choice;
} while (choice != 0);
```

- Body executes at least once — condition checked after
- Best for menus and input validation where you need to run once before checking

6.5 for vs while vs do-while

Loop	Use when
for	Known number of iterations
while	Condition-based, may not run
do-while	Must run at least once

6.6 break and continue

- `break` : immediately exits the loop — skips all remaining iterations
- `continue` : skips the current iteration and jumps to the next

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue; // skip even numbers
    if (i > 7) break;         // stop after 7
    cout << i << " ";
}
// Output: 1 3 5 7
```

6.7 Nested Loops

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

- Each time the outer loop runs once, the inner loop completes fully
-